

File Organizations:

Computerized databases are stored on physical media, which are organized into a storage hierarchy with two main categories:

▼ Primary Storage:

This is the computer's main memory and cache, which the CPU can access directly. It is characterized by **fast data access** but has **limited capacity** and is **more expensive**.

This is the highest level, directly used by the CPU.

- **Cache Memory:** The most expensive and fastest type, using static RAM (RAM). It is used by the CPU for techniques like prefetching to speed up execution.
- **Main Memory (DRAM):** The main work area for the CPU, holding program instructions and data. Its advantages are low cost, but it is slower than static RAM and is **volatile** (loses contents on power loss). Capacities are growing, with servers commonly having 8-16 GB, enabling **main memory databases** for real-time, fast-response applications.

▼ Secondary Storage:

This level includes larger, cheaper, but slower storage devices. Data here must be copied to primary storage for the CPU to process it.

This level includes larger, cheaper, but slower storage devices. Data here must be copied to primary storage for the CPU to process it.

- **Magnetic Disks:** The primary storage for large permanent databases.
- **Flash Memory:** An increasingly common form of nonvolatile memory between DRAM and magnetic disks. It uses EEPROM technology, offering fast access. A disadvantage is that an entire block must be erased and written at once. It is found in devices like USB drives, cameras, and phones.
- **Optical Disks:**
 - **CD-ROM:** Prerecorded, read-only optical storage.
 - **WORM Disks:** Write-Once-Read-Many disks, used for archiving.
 - **DVD:** Allows 4.5 to 15 GB of storage.
 - **Optical Jukeboxes:** Arrays of CD-ROM platters with high capacity (hundreds of GB) but slow retrieval, a technology in decline due to magnetic disk improvements.
- **Magnetic Tapes:** Used for archiving and backup. **Tape jukeboxes** are used as tertiary storage to hold terabytes of data (e.g., for satellite data archive)

▼ Do we use magnetic disks now ?

Magnetic disks (hard disk drives or HDDs) are still widely used today, though SSDs are on the rise.

Note :

SSDs → Electronic(Flash memory based)

HDDs → Magnetic

▼ Storage of Databases

▼ Importance of Understanding Storage and Organization

Understanding disk properties and data organization is critical for:

- **Database Designers & DBAs:** To choose the best storage techniques for performance and application requirements during **physical database design**.
- **DBMS Implementers:** To efficiently implement a range of data organization options within the DBMS.

▼ Data Access and File Organization

Typically, only a small portion of a database is processed at a time. This data is located on disk, copied to main memory for processing, and rewritten if changed. On disk, data is organized into **files of records**, where each record represents facts about entities, attributes, and relationships.

Primary file organizations determine the physical placement of records on disk and how they can be accessed. The main types are:

- **Heap File (Unordered File):** Records are stored in no particular order, with new records appended to the end.
- **Sorted File (Sequential File):** Records are kept ordered by the value of a specific field (the **sort key**).
- **Hashed File:** A **hash function** is applied to a specific field (the **hash key**) to determine a record's location on disk.
- **Other Organizations (e.g., B-trees):** Use tree structures for organization.

▼ Secondary Organizations

A **secondary organization** (also called an **auxiliary access structure** or **secondary index**) is a separate data structure that is built to provide efficient access to records in a database file based on a field that is **not** the primary organizing key.

Think of it as an **index at the back of a textbook (the list of terms used which are mentioned in the back of the book and where they are used)**.

- The main content of the textbook (the chapters) is organized in one primary way—sequentially by page number. This is like the **primary file organization** (e.g., sorted by an `employee_id`).
- The index at the back allows you to quickly find all the pages where a specific term (e.g., "photosynthesis") is mentioned, without having to read the entire book. This index is the **secondary organization**.

▼ Do we need them ?

Yes,

1. **To Ensure Performance for Common Queries:** Applications are built on a variety of queries. If your primary organization is on `customer_id`, but your users constantly search by `email` or `order_date`, those searches will be extremely inefficient without a secondary index. Indexes make these everyday operations fast. (in this case, email or order_date can be used as secondary index)

2. **To Avoid Full Table Scans**

3. **To Enforce Uniqueness on Non-Primary Keys:** If you have a column like `email` or `social_security_number` that must be unique, you create a **UNIQUE constraint**. This constraint is physically implemented as a secondary index. The database uses the index to quickly check if a value already exists when you try to insert a new record.
4. **To Speed Up Joins and Sorting:** When you join two tables on a common field, or when you use `ORDER BY` or `GROUP BY` on a column, the database can use a secondary index to perform these operations much more efficiently

▼ When You Might NOT Need a Secondary Organization

While essential for performance, there are cases where you might deliberately avoid creating a secondary index:

1. **On Very Small Tables:** If a table only has a few hundred rows, a full table scan is so fast that the overhead of maintaining an index isn't worth it.
2. **On Tables with Heavy Write Operations and Few Reads:** Indexes speed up **read** operations but slow down **write** operations (INSERT, UPDATE, DELETE). This is because every time data changes, the index must also be updated. On a table that is constantly being written to but rarely queried (e.g., a high-frequency logging table), the performance penalty of maintaining indexes might outweigh their benefit.
3. **On Columns that are Rarely Used in WHERE Clauses:** There's no benefit to indexing a column that you never search or filter on. It would just waste space and slow down writes.
4. **On Columns with Very Low Cardinality:** Cardinality refers to the number of unique values in a column. Indexing a column with very few distinct values (e.g., a `gender` column with only 'M', 'F', 'Other') is often not useful. The database might decide that a full table scan is faster than using an index that would point to a large percentage of the table anyway.

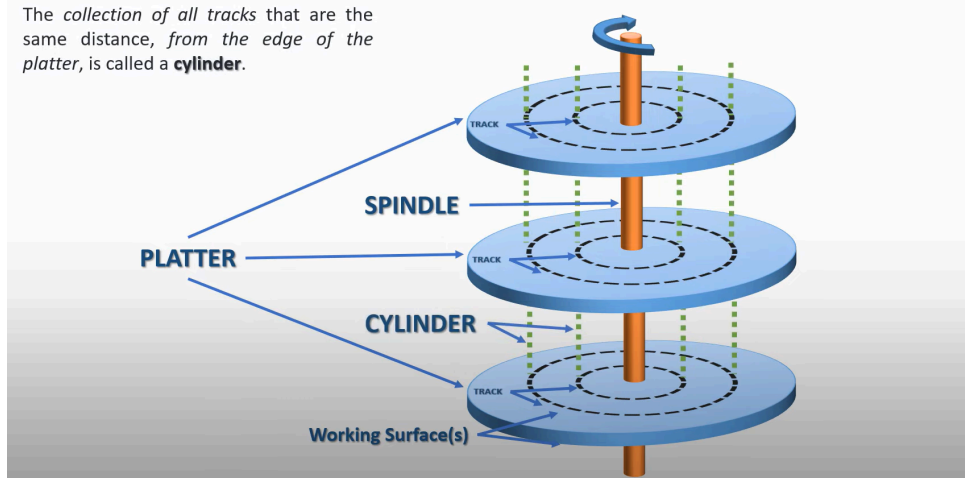
▼ Hardware Description of Disk Devices

▼ Basic Units of Data and Disk Capacity

- **Bit:** The most fundamental unit. A magnetized area on the disk represents either a 0 or a 1.
- **Byte (Character):** A group of bits (typically 8) that forms a single character of information.
- **Capacity:** The total number of bytes a disk can store. Capacities range from hundreds of Gigabytes (GB) to hundreds of Terabytes (TB) for large disk packs, and they continue to grow.

▼ Physical Structure and Organization

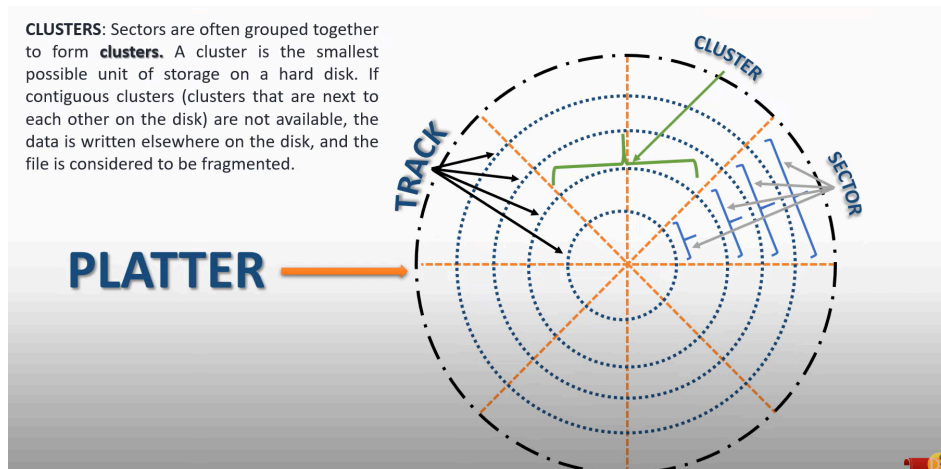
The collection of all tracks that are the same distance, from the edge of the platter, is called a **cylinder**.



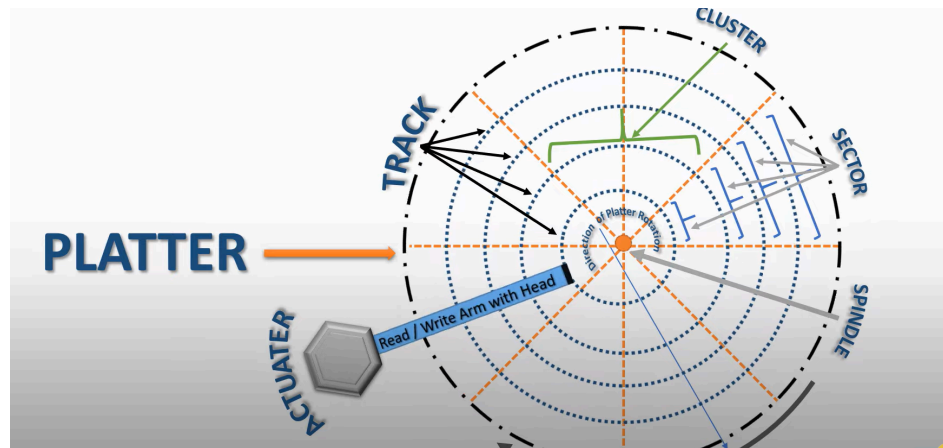
Disks are organized in a hierarchical physical structure to manage the vast amount of data they hold.

- **Platter:** The physical, circular magnetic disk itself. A disk can be single-sided or double-sided. Multiple platters are stacked to form a **disk pack**.
- **Track:** On each disk surface, data is stored in concentric circles. Each circle is called a **track**. The number of tracks ranges from hundreds to thousands.
- **Cylinder:** In a disk pack, the set of all tracks with the same diameter (one from each surface) forms a **cylinder**. This is a critical performance concept, as accessing data within the same cylinder is very fast.

CLUSTERS: Sectors are often grouped together to form **clusters**. A cluster is the smallest possible unit of storage on a hard disk. If contiguous clusters (clusters that are next to each other on the disk) are not available, the data is written elsewhere on the disk, and the file is considered to be fragmented.



- **Sector vs. Block (Page):**
 - **Sector:** A track is divided into smaller arcs called **sectors**. This division is often physically "hard-coded" onto the disk surface during manufacturing.
 - **Block (or Page):** The operating system groups sectors into logical **blocks** or **pages** during disk formatting. This is the standard unit of data transfer between the disk and main memory. Block sizes are fixed (typically 512B to 8KB) and are separated by **interblock gaps** containing control information.
- **Cluster:** Sometimes, several contiguous blocks are treated as a single unit for transfer, called a **cluster**.



▼ The Mechanics of Reading and Writing

The hardware that performs the actual data transfer is the **disk drive**.

- **Read/Write Head:** The electronic component that reads and writes bits magnetically. There is typically one head per disk surface.
- **Actuator Arm:** The mechanical arm that holds the read/write head. All arms move in unison, positioning all the heads over the same cylinder on their respective surfaces.
- **Disk Controller:** An embedded computer that interfaces between the disk drive and the main computer system (e.g., using SCSI). It accepts high-level commands and handles the precise mechanics of positioning the arm and activating the read/write heads.

There are two main types of disks:

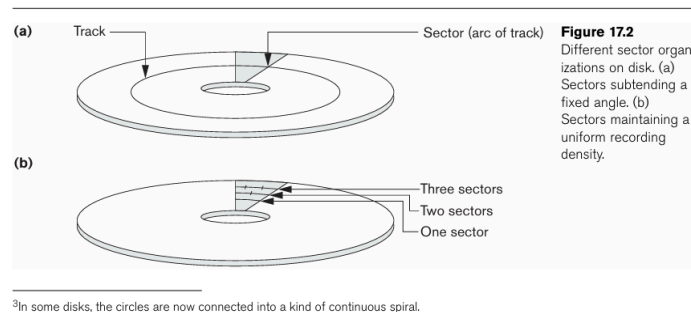
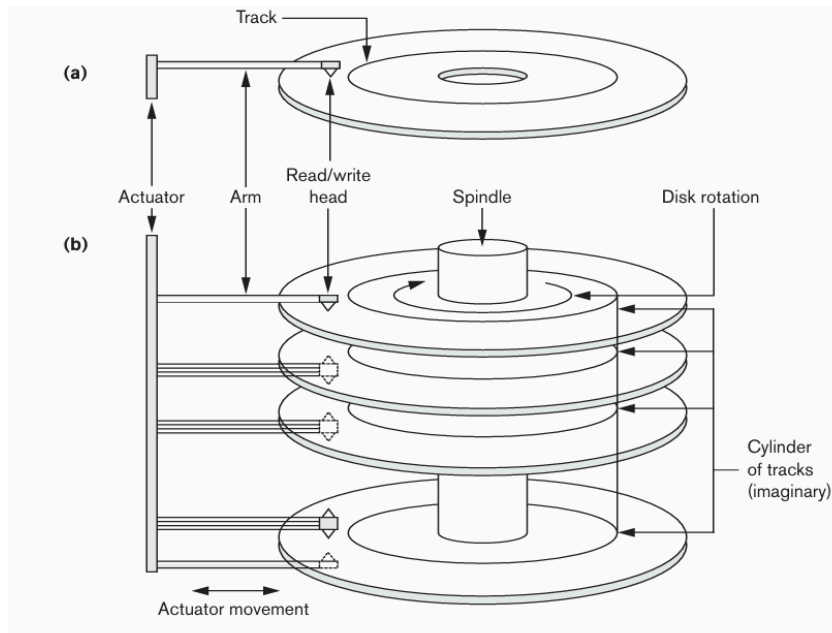
Movable-Head Disks (The Standard)

- **Heads per Surface:** One single read/write head for the entire surface.
- **How it works:** This one head is attached to a mechanical arm that must move **in and out** to position itself over the desired track. To read data from a different track, the arm must physically seek to that new location.

Fixed-Head Disks (The Rare, High-Performance Type)

- **Heads per Surface:** One read/write head for every track on the surface.
- **How it works:** The disk has an array of stationary heads, one permanently positioned over each track. There is **no moving arm**. To access a different track, the system simply **electronically switches** to the head that is already located over that track.

▼ Diagrams For Exam:



▼ The Cost of Access: Access Time

Accessing a random block of data is slow because it involves mechanical parts. The total time to access a disk block is the sum of three components:

1. **Seek Time:** The time for the actuator arm to move and position the read/write head over the correct **track**. This is the most time-consuming part, typically **5-10 milliseconds**.
2. **Rotational Delay (Latency):** Once on the right track, the disk must rotate until the beginning of the desired **block** moves under the read/write head. On average, this is the time for half a rotation. For a 15,000 RPM disk, this is about **2 milliseconds**.
3. **Block Transfer Time:** The time to actually read the data from the spinning disk and transfer it to the computer's memory once the head is in position. This is the fastest component, often taking less than a

millisecond.

The Key Takeaway: The total time to locate and read a random block is in the order of **milliseconds (e.g., 9-60 ms) which is slow compared to modern CPUs**

▼ The Solution: Optimizing for Contiguous Access

To overcome this bottleneck, the fundamental goal of database storage organization is to **minimize disk accesses**.

- **Bulk Transfer:** The most effective way to do this is to store "related information" on **contiguous blocks** (on the same track or cylinder).
- **Efficiency:** When reading multiple contiguous blocks, the system pays the high **seek time** and **rotational delay** only for the *first* block. Subsequent blocks can be read with near-zero delay, transferring at a very high **bulk transfer rate**.
- **Design Principle:** Therefore, the core principle of database file structures (like heaps, sorted files, and hashing) is to place data that will be accessed together physically close together on the disk to enable these efficient bulk transfers.

▼ Magnetic Tapes as Sequential Storage

Fundamental Access Method: Sequential vs. Random

- **Magnetic Disks:** Are **random access** devices. Any block can be accessed directly by its address.
- **Magnetic Tapes:** Are **sequential access** devices. To access the **n*th* block, the tape must physically scan through the preceding **n-1** blocks. This makes tape access inherently slower for retrieving specific data.
- Tapes store data on reels, similar to audio or videotapes, and require a **tape drive** to be read or written.

Primary Use Case: Backup and Archiving

Because of their sequential nature, tapes are not suitable for storing **online data** that requires frequent and immediate access. Instead, they serve two critical functions:

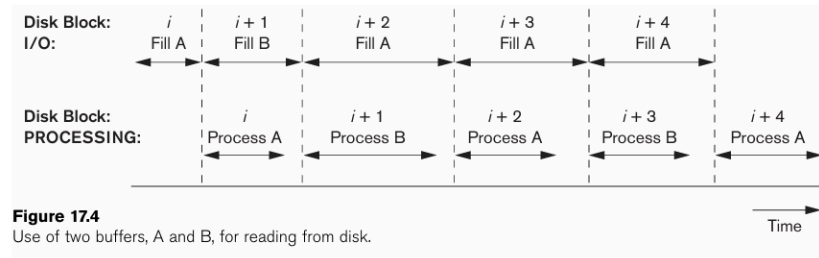
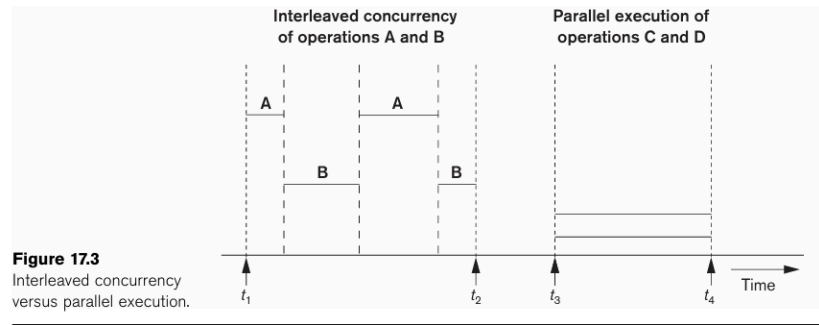
- **Backing Up Databases:** Tapes are a cost-effective medium for creating copies of disk files to protect against data loss from events like a **disk crash**.
- **Archiving:** They are used to store excessively large, seldom-used(very rarely used), or historical database files, keeping them available for long-term record-keeping but offline from active systems

▼ Buffering Of Blocks

The core problem is that disk I/O (Input/Output) is very slow compared to CPU processing speed. If the system had to wait for each block to be read before processing it, the CPU would sit idle for long periods.

The solution leverages **concurrency**—performing multiple tasks at the same time. This is made possible by two key factors:

1. **Independent Disk I/O Processor (Controller):** This is a specialized processor dedicated to managing disk operations. Once the CPU instructs it to read or write a block, the I/O processor handles the entire transfer independently, freeing up the main CPU to do other work.
2. **Multiple Buffers in Main Memory:** By reserving more than one area (buffer) in memory, the system can work on one block while simultaneously reading the next one into a different buffer.



This distinguishes between two types of concurrent execution.

- **Interleaved Concurrency (Processes A & B):** The CPU switches rapidly between two tasks (A and B) on a single processor. They are not running at the exact same instant, but their execution is interleaved over time.
- **Parallel Execution (Processes C & D):** Two tasks run simultaneously, at the exact same time. This requires multiple processors (e.g., the main CPU and the independent disk I/O processor).
- **Figure 17.4: Double Buffering:** This is the practical application of these concepts. The process is visualized on a timeline (t_1 , t_2 , etc.):
 - At time t_1 , the system starts reading **Disk Block i** into **Buffer A**.
 - At time t_2 , the read into Buffer A is complete. Now, two things happen **in parallel**:
 1. The CPU begins **processing** the data now available in **Buffer A**.
 2. The disk I/O processor simultaneously starts reading the next **Disk Block i+1** into **Buffer B**.
 - At time t_3 , the processing of Buffer A finishes, and the read into Buffer B finishes. The system then switches buffers:
 1. The CPU begins **processing** the data in **Buffer B**.
 2. The disk I/O processor starts reading **Disk Block i+2** back into the now-free **Buffer A**.
 - This cycle continues, creating a continuous, overlapping pipeline of data transfer and processing.

The CPU can start processing a block once its transfer to main memory is completed; at the same time, the disk I/O processor can be reading and transferring the next block into a different buffer. This technique is called **double buffering**.

▼ Records and Record Types

1. Fundamental Data Unit: Records

- Data is stored in the form of **records**.

- A record is a collection of related data values that together describe an **entity** (e.g., an EMPLOYEE) and its **attributes** (e.g., Name, Salary).

2. Structure of a Record: Fields and Types

- Each value in a record corresponds to a **field**.
- A **record type** (or record format) is defined by a collection of field names and their corresponding **data types**.
- The data type specifies what kind of values a field can hold and determines its storage size.

3. Common Data Types

- Standard programming data types are used, including:
 - **Numeric:** integer, long integer, floating point.
 - **String:** fixed-length or varying-length characters.
 - **Boolean:** TRUE/FALSE or 0/1 values.
 - **Special:** date and time.
- The storage size (in bytes) for these types is typically fixed by the system (e.g., 4 bytes for an integer, *k* bytes for a fixed-length string of *k* characters).

4. Handling Large Objects: BLOBs

- Some applications require storing very large, unstructured objects known as **BLOBs (Binary Large Objects)**, such as images, video, audio, or free text.
- Due to their size, BLOBs are not stored directly within the record. Instead, a **pointer** to the BLOB is stored in the record, and the BLOB data itself is stored separately in a pool of disk blocks.

▼ Files

▼ Intro

1. File Definition

- A **file** is defined as a sequence of records.

2. Fixed-Length vs. Variable-Length Records

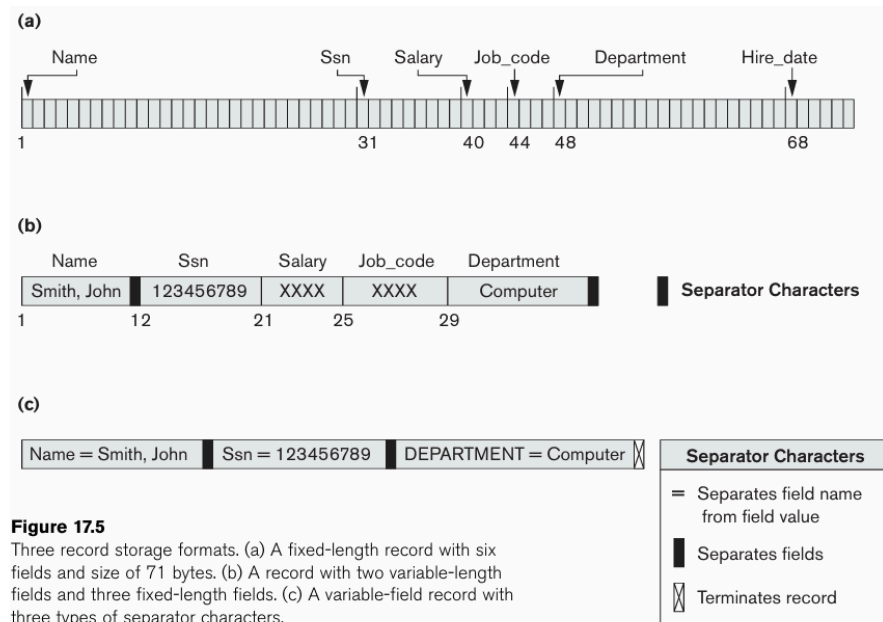
- **Fixed-Length Records:** Every record in the file has the exact same size (in bytes).
 - **Advantage:** Easy to process because the system can calculate the exact position of each field.
 - **Disadvantage:** Can lead to wasted space if used to simulate optional or repeating fields by reserving space for the maximum possible values.
- **Variable-Length Records:** Records in the file have different sizes.
 - **Advantage:** Saves storage space by allocating only the needed space for each record.
 - **Disadvantage:** More complex to process than fixed-length records.

3. Reasons for Variable-Length Records

A file has variable-length records for any of these four reasons:

1. **Variable-Length Fields:** One or more fields can have a varying size (e.g., a **Name** field).
2. **Repeating Fields:** A field can have multiple values for a single record (e.g., multiple phone numbers).

3. **Optional Fields:** A field may have a value for some records but not for others.
4. **Mixed File:** The file contains records of different types (and thus different formats/sizes) that are clustered together.



4. Storing Variable-Length Records

Several techniques are used to structure variable-length records so that fields can be identified:

- **Separator Characters:** Use special characters (like `?` or `%`) that do not appear in the data to mark the end of fields and the end of the record.
- **Field Lengths:** Precede each field's value with a number indicating its length in bytes.
- **Field-Name/Field-Value Pairs:** Store both the name and the value of each field in the record, separated by special characters. This is efficient when a record type has many fields but only a few are used in each record.
- **Field-Type Codes:** A more practical variation of the above, where a short code (like an integer) represents the field type instead of its full name.
- **Record Type Indicator:** For mixed files, each record is preceded by a code that identifies its type

▼ Record Blocking and Spanned versus Unspanned Records

1. The Need for Blocking

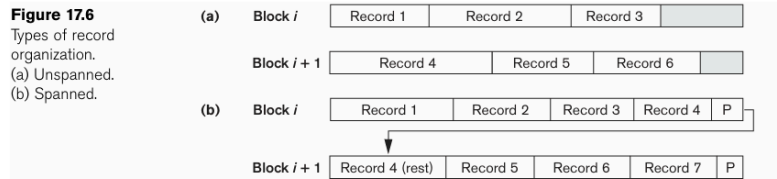
- Records must be grouped into **disk blocks** because the block is the fundamental unit of data transfer between disk and memory.

2. Blocking Factor

- For a file of fixed-length records, the number of records per block is calculated by the **blocking factor** (`bfr`).

- **Formula:** $bfr = \lfloor B / R \rfloor$ where B is the block size and R is the record size.
- This division often leaves **unused space** in the block, calculated as $B - (bfr * R)$ bytes.

3. Spanned vs. Unspanned Records



This concept defines whether a record is allowed to be split across multiple blocks.

- **Unspanned Organization:**
 - **Definition:** Records are **not allowed** to cross block boundaries. Each record is contained entirely within a single block.
 - **Use Case:** The default and simpler method, especially for **fixed-length records**. It simplifies record processing because each record starts at a known location within its block.
 - **Disadvantage:** Leads to unused space in blocks if B is not a perfect multiple of R .
- **Spanned Organization:**
 - **Definition:** Records **are allowed** to span more than one block. A record can start in one block and continue onto the next.
 - **Use Case:**
 1. **Mandatory:** When a single record is **larger than a block**.
 2. **Advantageous:** For variable-length records where the average record is large, as it reduces wasted space by utilizing the leftover space from the previous block.
 - **Mechanism:** A **pointer** at the end of the first block points to the block containing the remainder of the record.

4. Calculating Blocks for a File

- The number of blocks b needed for a file of r records is calculated using the blocking factor.
- **Formula:** $b = \lceil r / bfr \rceil$ blocks (the ceiling function is used to round up, ensuring space for all records).
- For variable-length records in a spanned organization, bfr represents the **average** number of records per block

▼ File Headers

1. Purpose of a File Header:

- A **file header** (or file descriptor) is a metadata structure that contains essential information about a file.
- It is used by system programs to correctly access and interpret the file's records.

2. Information Stored in the Header:

- **Disk Addresses:** The locations of the blocks that constitute the file.
- **Record Format Description:** The "blueprint" for how records are structured, which varies by record type:
 - For **Fixed-Length Records:** Field lengths and the order of fields.
 - For **Variable-Length Records:** Field type codes, separator characters, and record type codes.

3. The Process of Searching for a Record:

- The system copies one or more disk blocks into main memory buffers.
- Programs then search within these buffers for the desired record, using the format information from the file header to parse the records correctly.

4. The Performance Challenge and Goal:

- Without knowing the exact block address, a search may require a **linear search** through all file blocks, which is very slow for large files.
- The central goal of a good file organization is to **minimize the number of block transfers** needed to locate the block containing a desired record. This is the foundation for efficient indexing and hashing techniques.

▼ File Operations and Organizations

1. Types of File Operations

Operations on files are divided into two main categories:

- **Retrieval Operations:** Locate records to examine data (does not change the file).
- **Update Operations:** Change the file by inserting, deleting, or modifying records.

These operations use a **selection condition** (e.g., `Department = 'Research'`) to identify the target record(s). Complex conditions are broken down: a simple part is used to locate records on disk, and each located record is then checked against the full condition.

2. Record-at-a-Time Operations

These are low-level commands that work on a single record. A representative set includes:

- **Open.** Prepares the file for reading or writing. Allocates appropriate buffers (typically at least two) to hold file blocks from disk, and retrieves the file header. Sets the file pointer to the beginning of the file.
- **Reset.** Sets the file pointer of an open file to the beginning of the file.
- **Find (or Locate).** Searches for the first record that satisfies a search condition. Transfers the block containing that record into a main memory buffer (if it is not already there). The file pointer points to the record in the buffer and it becomes the current record. Sometimes, different verbs are used to indicate whether the located record is to be retrieved or updated.
- **Read (or Get).** Copies the current record from the buffer to a program variable in the user program. This command may also advance the current record pointer to the next record in the file, which may necessitate reading the next file block from disk.
- **FindNext.** Searches for the next record in the file that satisfies the search condition. Transfers the block containing that record into a main memory buffer (if it is not already there). The record is located in the buffer and becomes the current record. Various forms of FindNext (for example, Find Next record within

a current parent record, Find Next record of a given type, or Find Next record where a complex condition is met) are available in legacy DBMSs based on the hierarchical and network models.

- **Delete.** Deletes the current record and (eventually) updates the file on disk to reflect the deletion.
- **Modify.** Modifies some field values for the current record and (eventually) updates the file on disk to reflect the modification.
- **Insert.** Inserts a new record in the file by locating the block where the record is to be inserted, transferring that block into a main memory buffer (if it is not already there), writing the record into the buffer, and (eventually) writing the buffer to disk to reflect the insertion.
- **Close.** Completes the file access by releasing the buffers and performing any other needed cleanup operations.

Streamlined and Set-at-a-Time Operations

- **Scan.** If the file has just been opened or reset, Scan returns the first record; otherwise it returns the next record. If a condition is specified with the operation, the returned record is the first or next record satisfying the condition.
- **FindAll.** Locates all the records in the file that satisfy a search condition.
- **Find (or Locate) n.** Searches for the first record that satisfies a search condition and then continues to locate the next n-1 records satisfying the same condition. Transfers the blocks containing the n records to the main memory buffer (if not already there).
- **FindOrdered.** Retrieves all the records in the file in some specified order.
- **Reorganize.** Starts the reorganization process. As we shall see, some file organizations require periodic reorganization. An example is to reorder the file records by sorting them on a specified field.

3. Key Distinction: File Organization vs. Access Method

- **File Organization:** The physical layout of the data (how records and blocks are placed and linked on disk).
- **Access Method:** The set of operations (like Find, Insert) that can be applied to a file. An access method is built on top of a file organization.
- There is often a **trade-off**. For example, organizing records by **Ssn** for fast retrieval conflicts with organizing them by **Department** for efficient reporting.
- The designer must choose a compromise that efficiently supports the expected mix and importance of retrieval and update operations. Techniques like **ordering, hashing, and indexing** are used to achieve this.

▼ Unordered Files (Heap Files)

1. Core Concept & Use Cases

- **Structure:** Records are stored in the order they are inserted, with new records placed at the end of the file. This is the simplest file organization.
- **Common Uses:**
 - As a base for **additional access paths** like secondary indexes.
 - For collecting and storing data records for future processing.

2. Performance of Operations

- **Insertion: Highly efficient.** The last disk block is loaded into a buffer, the new record is added, and the block is written back. The file header keeps the address of the last block to facilitate this.
- **Search (Retrieval): Very inefficient.** Requires a **linear search** through the file block by block.
 - On average, finding a single record requires searching **half the file blocks (b/2)**.
 - If multiple or no records match, all **b blocks** must be searched.
- **Reading in Sorted Order:** Requires creating a sorted copy of the file, which is an expensive operation.

3. Record Deletion & Storage Management

- **Process:** A record is found, its block is loaded into a buffer, the record is removed (or marked), and the block is rewritten.
- **Deletion Techniques:**
 1. **Physical Deletion:** Removing the record from the block, which leaves **unused space** and wastes storage.
 2. **Logical Deletion:** Using a **deletion marker** (a special byte or bit) to mark a record as deleted. Search programs then ignore marked records.
- **Storage Reclamation:** Both techniques require **periodic file reorganization** to reclaim space by packing records and removing deletions. Alternatively, extra bookkeeping can track empty slots for new insertions.
- **Modification:** Modifying a variable-length record may require a delete and re-insert if it no longer fits in its original space.
- **Direct Access:** For **fixed-length, unspanned records**, any record can be accessed directly by its position (the *i* th record). A file that enables this is called a **relative** or **direct file**. This is useful for building indexes but does not help with search conditions based on field values.

▼ Why it does not help with search conditions based on field values ?

The Analogy: A Phone Book vs. A Packing List

Imagine a warehouse full of boxes.

- **The Direct File (Relative File)** is like having a **packing list** for the warehouse. The list says:
 - "Box 1 is on Shelf A."
 - "Box 2 is on Shelf B."
 - "Box 3 is on Shelf C."
- **A Search Condition** is like asking: **"Find the box that contains a red toaster."**

Why the Packing List (Direct Access) Fails for the Search:

The packing list is organized by *box number*, not by the *contents* of the boxes. To find the red toaster, you have no choice but to go to Shelf A, open Box 1, and look through everything inside. Then go to Shelf B, open Box 2, and look through everything. You would have to repeat this for every single box until you found it. This is a **linear search**.

▼ Ordered Files (Sorted Files)

An ordered file is a file where the records are **physically arranged on disk** in a specific sequence based on the value of one field, known as the **ordering field**. If this field is also a unique key (like **Ssn**), it's called the **ordering key**.

Summary of Important Points

1. Advantages (Efficient Operations)

- **Ordered Retrieval:** Reading all records in the order of the ordering key is extremely efficient, as the records are already stored in that sequence. This allows for the use of techniques like double buffering for continuous reading.
- **Fast Search on Ordering Key:**
 - **Binary Search:** A specific record can be found by its ordering key value using a binary search on the file blocks. This reduces the average number of block accesses from **$b/2$** (linear search) to **$\log_2(b)$** , a massive improvement for large files.
 - **Range Queries:** Search conditions using **>**, **<**, **≥**, and **≤** on the ordering field are very efficient because all qualifying records are stored contiguously in the file.
- **Next Record Access:** Retrieving the next record in sequence is very fast, as it is usually located in the same disk block.

2. Disadvantages (Inefficient Operations)

- **Inefficient Insertion and Deletion:**
 - **Insertion:** To maintain the physical order, inserting a new record requires finding its correct position and then **shifting roughly half of the records** on average to make space. This is a very expensive operation.
 - **Deletion:** Similar to heap files, deletion leaves wasted space and requires periodic reorganization.
- **Poor Performance on Non-Ordering Fields:**
 - Any search, access, or ordered retrieval that is based on a field **other than the ordering field** degrades to a slow **linear search** of the entire file. The physical order provides no benefit for these queries.
- **Complex Record Modification:**
 - Modifying a **non-ordering field** is straightforward if the record can be found.
 - Modifying the **ordering field** itself is treated as a **delete** of the old record followed by an **insert** of the new one, incurring the high cost of both operations.

3. Optimization Techniques for Insertions

To mitigate the high cost of insertions, two main techniques are used:

1. Overflow (Transaction) File:

- Another frequently used method is to create a temporary unordered file called an overflow or transaction file. With this technique, the actual ordered file is called the main or master file. New records are inserted at the end of the overflow file rather than in their correct position in the main file. Periodically, the overflow file is sorted and merged with the master file during file reorganization.

- **Pro:** Makes insertions very fast.
- **Con:** Makes searching more complex, as a record not found in the main file via binary search requires a subsequent linear search of the overflow file.
- Periodically, the overflow file is sorted and merged with the main file in a **reorganization** process.

2. Free Space in Blocks:

- Each block is left partially empty to accommodate new records, reducing the need for immediate shifting.
- This is only a temporary solution; once the free space is used, the original insertion problem returns.

▼ Hashing Techniques

1. Core Purpose & Function

- **Purpose:** To provide **very fast, direct access** to a specific record.
- **Mechanism:** Uses a **hash function (h)** to directly compute the disk address of a record.
- **Process:** The hash function takes the value of a record's **hash field** and converts it into the **address of the disk block** where the record should be stored.

2. Key Terminology

- **Hash File:** The file organized using hashing.
- **Hash Field:** The field used for the search condition. The search must be an **equality condition** (e.g., `Ssn = '123-45-6789'`).
- **Hash Key:** If the hash field is also a **key field** (guaranteed to be unique), it is called the hash key.
- **Hash Function:** The function that maps a hash field value to a physical disk block address.

3. Performance Advantage

- The primary advantage is **speed**. In the ideal case, retrieving a record requires only **a single disk block access**, regardless of the file size. This is significantly faster than the linear search of a heap file ($b/2$ accesses) or the binary search of an ordered file ($\log_2(b)$ accesses).

4. Scope of Application

- Hashing is used for both:
 1. **Internal Files:** As a search structure within a program.
 2. **External Files:** For organizing records on a disk, which requires modifications to handle the constraints of disk storage.

▼ Internal Hashing

1. Basic Structure

- Implemented as a **hash table**—an array of `M` slots (from index `0` to `M-1`).
- A **hash function** transforms a record's hash field value into an integer within this array's index range, which becomes the record's storage address.

2. Hash Functions

The goal is to uniformly distribute records across the hash table. Common techniques include:

- **Division-Remainder (mod):** $h(K) = K \bmod M$ (most common).
- **Folding:** Splits the key into parts and combines them (e.g., addition, XOR).
- **Digit Extraction:** Selects specific digits from the key to form the address.
- **String Conversion:** Converts character strings to integers by manipulating their ASCII codes.

3. The Collision Problem

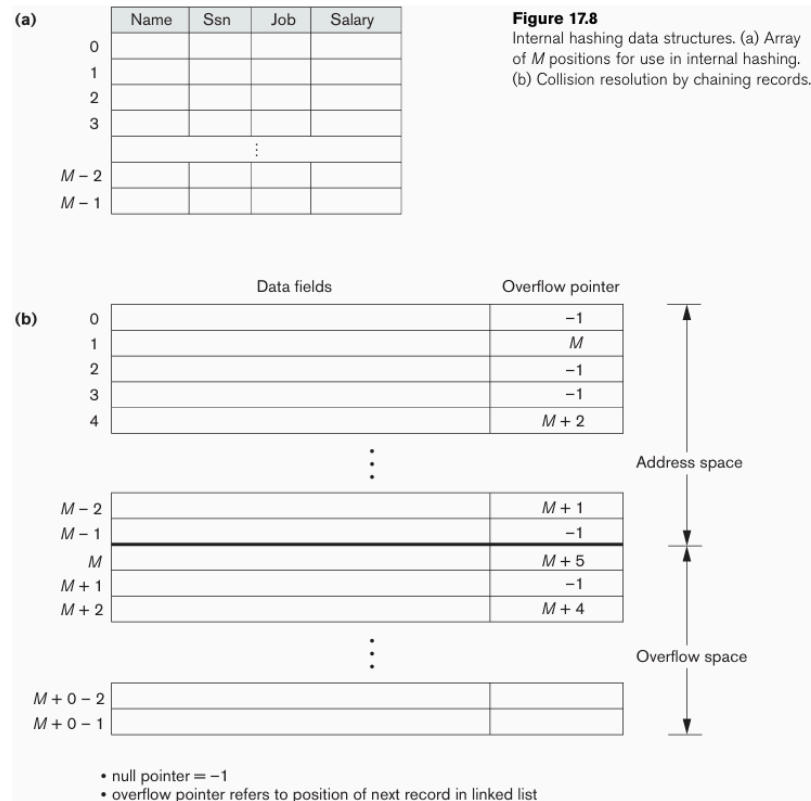
- **Collision:** Occurs when two different hash field values are mapped by the hash function to the **same address**.
- **Cause:** The potential number of unique hash field values (hash field space) is much larger than the number of available table slots (address space). Collisions are inevitable.

4. Collision Resolution Methods

- **Open Addressing:**
 - When a collision occurs, the algorithm sequentially probes subsequent slots until an empty one is found.
 - **Disadvantage:** Deletion is complex, and clusters of occupied slots can form, reducing performance.
- **Chaining:**
 - Each table slot points to a linked list (chain) of records.
 - Colliding records are placed in an overflow area and linked to their hash address.
 - **Advantage:** Simple and efficient for insertion and deletion.
- **Multiple Hashing:**
 - Uses a second (or third) hash function if the first one causes a collision, eventually falling back to open addressing.

5. Design Considerations for Efficiency

- **Load Factor:** The ratio of stored records to table size (r/M). For optimal performance, the hash table should be kept **70-90% full**.
 - Too high: Excessive collisions slow down access.
 - Too low: Wastes memory space.
- **Table Size (M):** Often chosen to be a **prime number** to improve distribution when using the mod function.



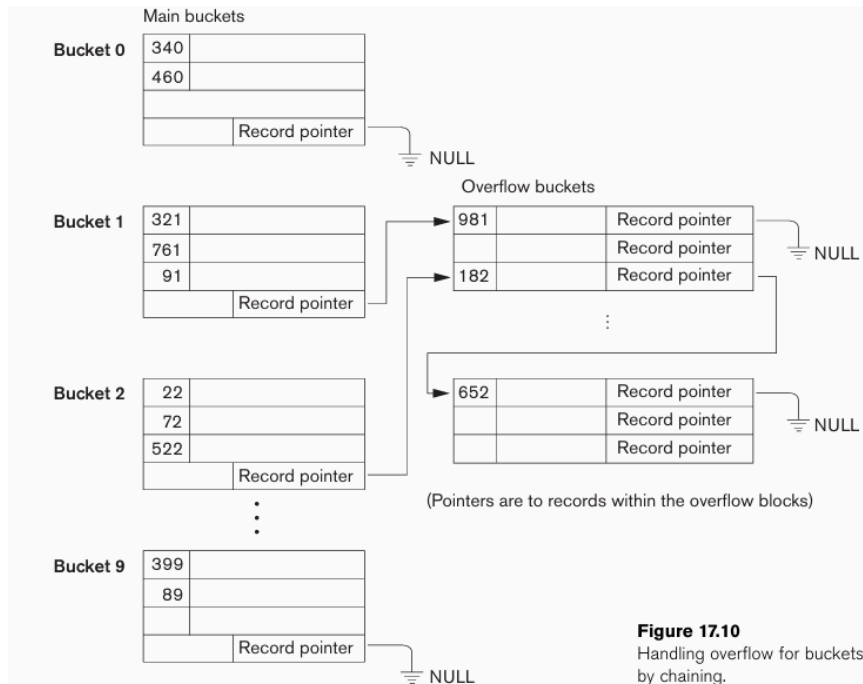
▼ External Hashing

1. Core Concept & Structure

- **Purpose:** Designed for **disk files**, optimized to minimize disk I/O.
- **Basic Unit:** The **bucket**, which is one disk block or a cluster of contiguous blocks.
- **Addressing:** The hash function maps a key to a **relative bucket number**. A separate table in the file header maps this number to the actual **disk block address**.

2. Collision Handling

- **Bucket Advantage:** Multiple records can hash to the same bucket without issue until the bucket reaches its capacity.
- **Overflow Handling:** When a bucket is full, new records that hash to it are placed in **overflow buckets**. These are chained together in a linked list, with each bucket containing a pointer to its overflow chain.
- **Record Pointers:** Pointers in the overflow chain are **record pointers**, containing both a block address and the record's position within that block



3. Performance & Limitations

- **Fast Retrieval:** Provides the **fastest possible access** for retrieving a record when searching by its hash field value (often just one disk access).
- **Static Hashing Problem:** The basic scheme uses a **fixed number of buckets (M)**, which is inefficient for dynamically growing files. It can lead to either wasted space or excessive overflow chains.
- **Inefficient for Other Searches:** Searching by a **non-hash field** requires a slow **linear search** of the entire file, just like an unordered heap file.

4. Record Operations

- **Deletion:** A record is removed from its bucket or overflow chain. The space can be reused, often tracked by a list of free overflow locations.
- **Modification:**
 - Changing a **non-hash field** is simple: the record is updated in place.
 - Changing the **hash field** is complex: it requires **deleting the old record** and **inserting a new one**, as the hash value (and thus the bucket) changes.

▼ Order-Preserving Hashing

- Some special hash functions maintain the order of records. This is rare and typically only works in specific scenarios where keys are generated in sequence (e.g., using an invoice number directly as the bucket address).

▼ Static Hashing(The Techniques discussed up until now) vs. Dynamic Hashing

Static Hashing

In static hashing, the number of buckets is **fixed** when the database is created.

- **Advantages:**
 - **Simplicity:** Easy to implement and understand.
- **Disadvantages:**
 1. **Fixed Size Problem:** The maximum number of records is roughly $M * m$ (where m is the bucket capacity).
 2. Space Inefficiency

Dynamic Hashing (Extendible Hashing)

In dynamic hashing, the number of buckets can **grow or shrink** dynamically as records are inserted and deleted.

▼ Extendible Hashing

Extendible Hashing uses a two-level structure to achieve dynamic growth: a **directory** and the **buckets** themselves.

1. Core Components

- **Directory:**
 - An array of 2^d pointers (where d is the **global depth**).
 - The first d bits of a record's hash value are used as an **index** into this directory to find which bucket it belongs in.
- **Buckets:**
 - The actual disk blocks that store the records.
 - Each bucket has a **local depth (d')**, which indicates how many hash bits are currently used to determine which records belong in that specific bucket.

2. How It Works (The Process)

- **Initial State:** All directory entries might point to a single bucket. The global depth d and the local depth d' of that bucket are both 0.
- **Insertion:**
 1. Hash the record's key to get a bit string.
 2. Use the first d bits to look up the directory entry, which points to the target bucket.
 3. If the bucket has space, insert the record.
- **Handling Overflow (Bucket Splitting):**
 1. If the target bucket is full, it is **split**.
 2. The bucket's **local depth (d')** is increased by 1.
 3. The records in the overflowing bucket are rehashed. Their distribution between the old and new bucket is now based on the **first d' bits** (one more bit than before).
 4. The directory entries that pointed to the old bucket are updated. Some will now point to the new bucket.

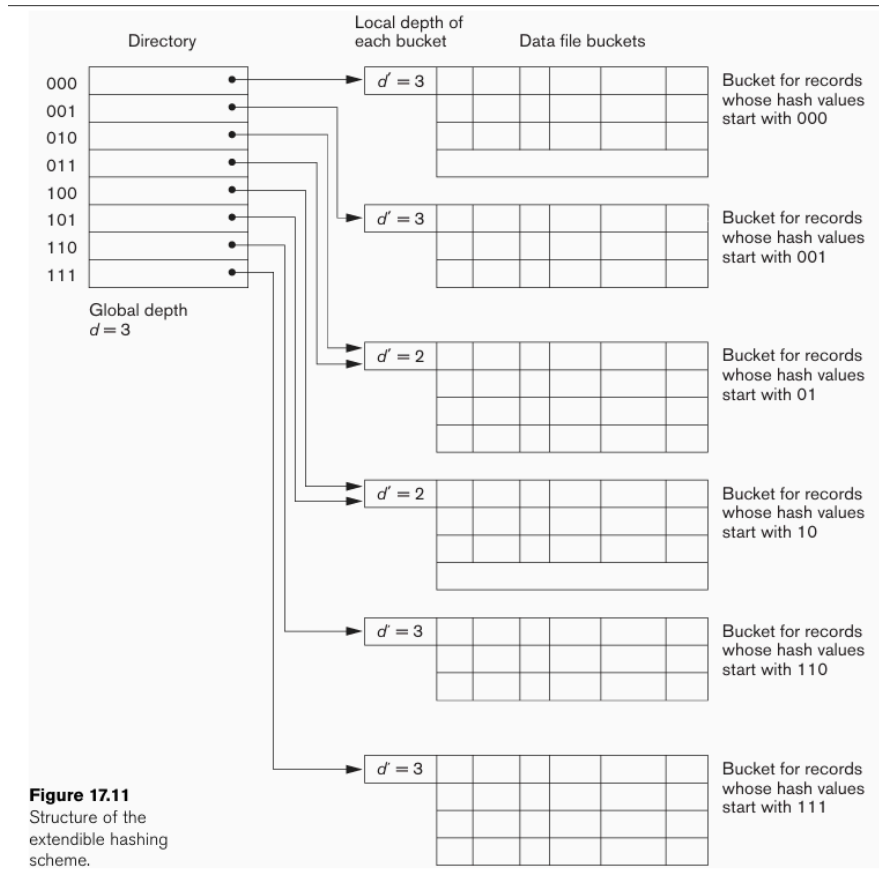
3. Directory Doubling (The "Extendible" Part)

A crucial event occurs when a bucket that needs to be split has a **local depth equal to the global depth** ($d' == d$).

- In this case, simply splitting the bucket isn't enough because the directory isn't large enough to have unique entries for the two new buckets.
- The solution is to **double the directory size**:
 - The **global depth d is increased by 1** (e.g., from 3 to 4).
 - The number of directory entries **doubles** (from $2^3=8$ to $2^4=16$).
 - Each original directory entry is effectively split into two, both initially pointing to the same bucket as before.
 - Now, the directory has the necessary granularity to point to the two new buckets created by the split.

4. Advantages & Trade-offs

- **Advantages:**
 - **No Performance Degradation:** Overflow chains are avoided through splitting, so retrieval stays fast.
 - **Dynamic Growth:** The file can grow indefinitely.
 - **Localized Reorganization:** Most splits only affect one bucket. Full-file reorganization is never needed.
 - **Negligible Directory Overhead:** The directory is small and stored in memory.
- **Trade-off (Disadvantage):**
 - **Extra Block Access:** Retrieving a record typically requires **two disk accesses**: one to read the directory (usually cached in RAM) and one to read the actual bucket. This is a minor penalty compared to the performance collapse of a static hash file with long overflow chains.



The **local depth** (d') is a crucial piece of metadata stored with each bucket. It tells you:

"How many hash bits are currently used to decide which records belong in this specific bucket."

In simple terms, the global depth d tells you:

"How many bits of the hash value we are currently using to look up a bucket address in the directory."

▼ Dynamic and Linear Hashing

While Extendible Hashing uses a flat directory, other methods were developed to handle file growth dynamically. The two main alternatives are:

1. **Dynamic Hashing (Tree-Based):** Uses a binary tree structure as a directory.
2. **Linear Hashing:** Grows the file without any directory by splitting buckets in a fixed, linear order.

▼ Dynamic Hashing (Tree-Based)

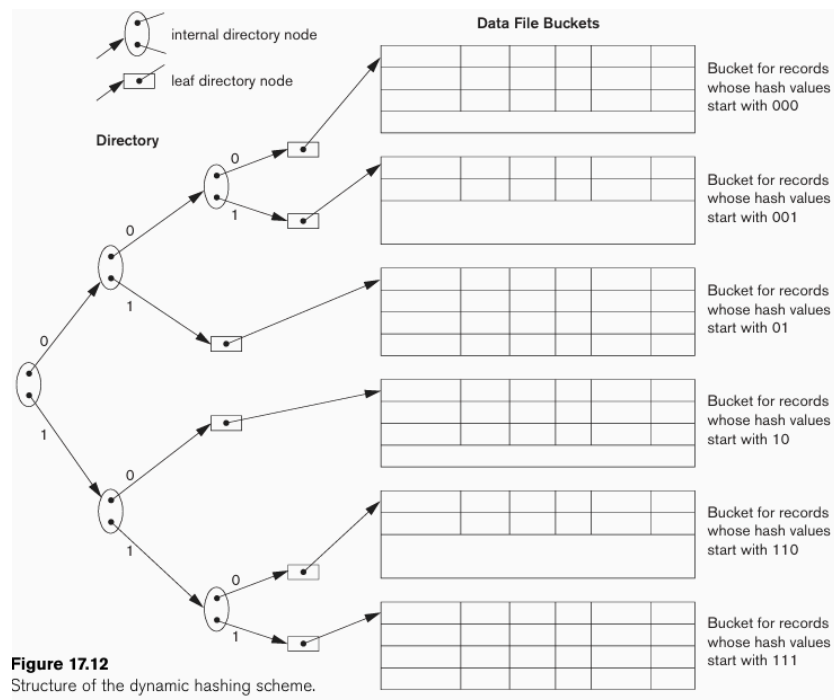
<https://youtu.be/r4GkXtH1la8?si=cxPX4sq-miwXS7KX>

This is a precursor to Extendible Hashing that uses a tree instead of a flat array.

- **Core Structure:**

- **Internal Nodes:** Act as decision points. They have two pointers: a *left* pointer for a 0 bit and a *right* pointer for a 1 bit in the hash value.
- **Leaf Nodes:** Point to the actual data buckets on disk.

- The "directory" is this implicit tree structure.
- **How it Works:**
 - To find a record, the system traverses the tree from the root, using each bit of the hash value to go left (0) or right (1), until it reaches a leaf node that points to the correct bucket.
 - When a bucket overflows, the tree is grown by replacing a leaf node with a new internal node, which then points to two new leaf nodes (the split bucket and the new one). This is similar to how buckets split in Extendible Hashing, but managed by a tree.
- **Comparison to Extendible Hashing:**
 - The eventual distribution of records into buckets is very similar.
 - The key difference is the **directory organization** (tree vs. flat array).



▼ Linear Hashing

https://youtu.be/h37Jhr21ByQ?si=MxKL_g21NZ1elfqu

Terminology	Descriptions
$h_0, h_1, h_2, \dots$	a family of hash functions, where each function's range is twice of its predecessor
N	Initial number of buckets
d_0	The number of bits to represent N
Level	Indicate the number of split cycle completed, initially 0
Next	Pointer to the next bucket inline to be split, initially points to the first bucket in the table.

Properties

$$N = 2^{d_0}, \text{ for some } d_0$$

$$d_i = d_0 + i$$

$$h_i(\text{key}) = h(\text{key}) \bmod(2^i N)$$

Splitting Criterion

- By Load Factor

- Load factor = $\frac{\text{\# of entries}}{(\text{\# of buckets}) \cdot (\text{bucket capacity})}$

- Load factor = $\frac{3}{2 \cdot 2} = 75\%$

Bucket capacity = 2
Load factor = 75%

B(0)	2	4
B(1)	3	

This is a major technique that allows a file to grow and shrink without a directory.

Core Concept: Buckets are split in a fixed, linear order (0, 1, 2, 3, ...) regardless of which bucket actually overflowed. An overflow in any bucket triggers the split of the next bucket in line.

How it Works:

Initial State: Start with M buckets, using hash function $h_i(K) = K \bmod M$.

The Split Trigger: An overflow occurs in any bucket.

The Split Action: The bucket at the current "split pointer" (n) is split. For example, when $n = 0$, bucket 0 is split into the original bucket 0 and a new bucket M at the end of the file.

Redistribution: Records from the split bucket are redistributed using a new hash function, $h_{\{i+1\}}(K) = K \bmod 2M$. A crucial property is that $h_{\{i+1\}}$ will correctly redistribute the records that were in the split bucket under the old function h_i .

The Split Pointer: A value n (from 0 to M-1) keeps track of the next bucket to be split. After a split, n is incremented.

Searching for a Record (Algorithm 17.3):

Apply the original hash function, $h_i(K)$, to get a bucket number m.

If m is less than the current split pointer n, it means that bucket has already been split. Therefore, you must use the new hash function $h_{\{i+1\}}(K)$ to find the record, as it could be in the original bucket m or the new bucket it was split into.

If $m \geq n$, the bucket hasn't been split yet, so the record is in bucket m (or its overflow chain).

Advantages and Control:

No Directory: Eliminates the space and one-level-of-indirection overhead of Extendible Hashing.

Controlled by Load Factor: Splits can be triggered based on the file's overall load factor ($l = \text{number_of_records} / (\text{bucket_capacity} \cdot \text{number_of_buckets})$) rather than on every single overflow. This allows for better performance tuning.

Dynamic Shrinking: The process can also work in reverse; if the load factor falls below a threshold, buckets can be merged linearly.

▼ Files of Mixed Records and Alternative Organizations

1. Files of Mixed Records

The Problem:

Traditional file organizations (heap, sorted, hashed) store records of only one type (e.g., only EMPLOYEE records). However, databases have multiple, related entities (e.g., STUDENTS and DEPARTMENTS). Relationships are typically implemented **logically** using connecting fields (like a `Major_dept` field in a STUDENT record that points to a DEPARTMENT record). To get data from both, you must perform two separate lookups, which can be slow.

The Solution: Physical Clustering in Mixed Files

A **mixed file** stores records of **different types** in the same physical file area to optimize performance for frequently used relationships.

- **How it Works:**
 - Records from different types (e.g., a DEPARTMENT record and all its related STUDENT records) are placed **contiguously on disk** (this is called **physical clustering**).
 - To distinguish one record type from another, each record has a **record type field** as its first field. The DBMS uses this to interpret the rest of the record's structure based on the system catalog.
- **Why Use It? Performance.**
 - If a common query is "Retrieve a DEPARTMENT and all its STUDENTS," a mixed file can answer this with minimal disk I/O. Instead of reading the DEPARTMENT block and then seeking to various other blocks for each STUDENT, the DBMS can read one contiguous section of the disk.
- **Common In:**
 - **Legacy Systems:** Hierarchical and Network DBMSs.
 - **Object DBMSs:** To store complex, related objects together.

2. B-Trees and Other Data Structures as Primary Organization

The Concept:

The primary file organization (how records are placed on disk) doesn't *have* to be heap, sorted, or hashed. It can be any disk-adapted data structure.

- **B-Tree as a Primary Organization:**
 - Typically, B-Trees are used for *indexing* (a secondary access path). However, for small files, some DBMSs use the B-Tree structure itself to physically order and store the actual data records.
 - This means the records themselves are stored in the leaf nodes of the B-Tree, providing both physical ordering and efficient access.
- **Column-Based Storage:**

- This is a radical alternative to the traditional **row-based storage** (where all fields of a record are stored together).
- **How it works:** Instead of storing all of `Employee1`'s data together, then all of `Employee2`'s data, it stores *all values of a single column* together.
 - One disk block might contain all `Salary` values.
 - Another block might contain all `Name` values.
- **Advantage:** Extremely efficient for data warehousing and analytical queries that aggregate data from a few columns over millions of records (e.g., "find the average salary"). It only needs to read the relevant column blocks, not entire rows.
- **Disadvantage:** Very inefficient for transactional queries that need to insert or update an entire row.

▼ Parallelizing Disk Access Using RAID Technology

1. The Problem: The Performance Gap

- There is a massive and growing performance gap between processors/RAM (improving very rapidly) and disk drives (improving slowly, especially in access time).
- This creates a bottleneck for data-intensive applications.

2. The Solution: RAID

- **RAID** (Redundant Arrays of *Inexpensive* or *Independent* Disks) uses an array of many small, cheap disks to act as a single, high-performance logical disk.
- Its main goals are:
 1. **Improved Performance** via parallelism.
 2. **Improved Reliability** through data redundancy.

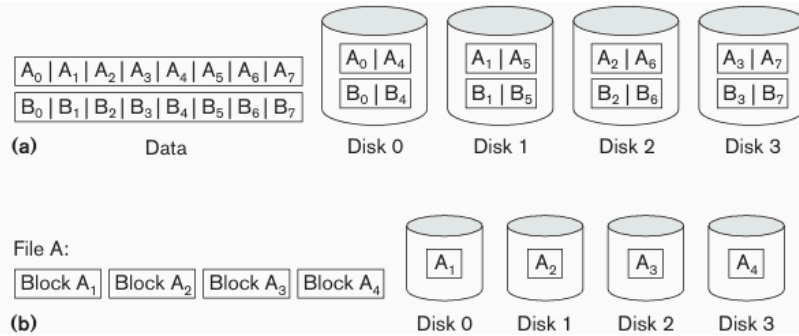
3. The Key Technique: Data Striping

- **Data Striping** is the fundamental method for achieving high performance.
- It transparently distributes data across multiple disks, making them appear as one large, fast disk.
- **How it works:** A file is split into segments (bits or blocks), and these segments are written across the disks in the array concurrently.
- **Benefits:**
 - **Parallel I/O:** Multiple I/O operations can be serviced simultaneously by different disks.
 - **High Transfer Rates:** Reading or writing a large file is faster as all disks work on their portion at the same time.
 - **Load Balancing:** Work is distributed evenly across all disks.

4. Achieving Reliability: Redundancy

- To protect against disk failures, RAID systems store **redundant information** (such as parity or error-correcting codes) on the disks.
- If one disk fails, the data on it can be reconstructed from the remaining data and the redundant information stored on the other disks.

Figure 17.13
Striping of data
across multiple disks.
(a) Bit-level striping
across four disks.
(b) Block-level
striping across four disks.



¹²This was predicted by Gordon Bell to be about 40 percent every year between 1974 and 1984 and is now supposed to exceed 50 percent per year.

▼ Formula

1. Blocking Factor $bfr = \lceil B / R \rceil$ where B is the block size and R is the record size.
2. The number of blocks b needed for a file of r records is calculated using the blocking factor. $b = \lceil r / bfr \rceil$ blocks (the ceiling function is used to round up, ensuring space for all records). For variable-length records in a spanned organization, bfr represents the **average** number of records per block

▼ RAID

◆ 1. Background: Why RAID?

As processors and RAM have become **faster** and **larger**, **disk performance** (especially access time) has **not kept pace**.

- CPU speed → increases rapidly.
- Disk capacity → increases quickly.
- Disk **access time and transfer rate** → improve **very slowly** (~10–20% per year).

This mismatch means:

- The CPU often sits **idle**, waiting for disk I/O.
- Reliability decreases when using **many disks**, since each adds a failure point.

👉 RAID was introduced to address **performance** and **reliability**.

◆ 2. What is RAID?

RAID (Redundant Arrays of Independent Disks) is a **technology** that combines multiple small disks to:

- Act as a **single logical storage unit**, and
- Achieve:
 1. **Higher performance** (through *parallelism*), and

2. **Higher reliability** (through *redundancy*).

◆ 3. Key Idea — Data Striping

RAID uses a concept called **data striping**.

► Definition

Data striping means **splitting the data** into smaller parts (bits, bytes, or blocks) and distributing them **across multiple disks**.

► Why?

Because if data is spread across disks, **many disks can read/write simultaneously**, increasing the total I/O throughput.

► Two Main Types:

(a) Bit-level Striping

- Each byte's bits are spread across disks.
- Example: Bit 0 → Disk 0, Bit 1 → Disk 1, Bit 2 → Disk 2, etc.
- All disks participate in every I/O.
- Improves **transfer rate** for large sequential data.
- Shown in Figure 17.13(a).

(b) Block-level Striping

- Whole blocks (like 4 KB or 8 KB chunks) are stored on different disks.
- Example:
 - Block A1 → Disk 0
 - Block A2 → Disk 1
 - Block A3 → Disk 2
 - Block A4 → Disk 3
- Improves **parallelism for small reads/writes**, because multiple requests can be served at once.
- Shown in Figure 17.13(b).

◆ 4. RAID Goals

Goal	Technique	Benefit
Reliability (Fault Tolerance)	Redundancy (Mirroring, Parity, Error Correction)	Survive disk failures
Performance	Data Striping (Bit or Block)	Parallel read/write operations

◆ 5. Reliability: Handling Disk Failures

If you have many disks, failures become more likely.

For example:

If one disk has a **Mean Time Between Failures (MTBF)** of 200,000 hours,
then 100 such disks have $MTBF \approx 200,000 / 100 = 2,000 \text{ hours } (\approx 83 \text{ days})$.
To tolerate failures, RAID uses **redundant storage**.

► Techniques for Reliability:

(a) Mirroring (RAID 1)

- Each disk has an **exact duplicate** (shadow).
- Data is written to both.
- If one fails, the other continues.
- Improves read speed (read from whichever is free).
- Very high reliability:

$$MTTDL = (MTBF)^2 \times \text{Repair Time}$$

$$\text{MTTDL} = 2 \times \text{Repair Time} (MTBF)^2$$

(b) Parity and Error-Correcting Codes

- Instead of full duplication, store **extra computed data** (e.g., parity bits).
- When a disk fails, its data is **reconstructed** using parity.
- Example:
If $A \oplus B \oplus C = \text{Parity}$,
and Disk C fails $\rightarrow C = A \oplus B \oplus \text{Parity}$.
- Uses less extra space than mirroring.

⚙️ What RAID Is

RAID (Redundant Array of Independent Disks) combines multiple disks into a single logical storage system to achieve:

1. **Performance improvement** (by parallelizing reads/writes)
2. **Reliability** (by storing redundant data)

RAID can be implemented using hardware controllers or software in the operating system.

◆ RAID Level 0 – *Striping (No Redundancy)*

◆ Concept:

- Data is **split (striped)** across multiple disks.
- No redundancy $\rightarrow$ if any disk fails, all data is lost.

◆ Example (4 disks):

Disk 0	Disk 1	Disk 2	Disk 3
A1	A2	A3	A4
B1	B2	B3	B4

♦ **Features:**

- Very high **read/write performance** (parallelism)
- **No fault tolerance**
- **Full capacity usable**

♦ **Used for:**

- Non-critical data (temporary files, cache, gaming, etc.)
-

♦ **RAID Level 1 – *Mirroring***

♦ **Concept:**

- Each disk has an **exact mirror copy**.
- Data written to one disk is also written to its mirror.

♦ **Example (2 disks):**

Disk 0	Disk 1
A	A
B	B

♦ **Features:**

- **100% redundancy** (fault tolerant — can survive one disk failure)
- **Read speed doubles** (read from either disk)
- **Write speed same** (data written to both)
- **Storage efficiency = 50%**

♦ **Used for:**

- **Critical systems** (banking, logs, servers, OS drives)
-

♦ **RAID Level 2 – *Hamming Code (Bit-level ECC)***

♦ **Concept:**

- Data is striped at **bit level**.
- Redundant disks store **error-correcting codes (Hamming code)**.
- Can detect and correct single-bit errors.

♦ **Example:**

- If you have 4 data disks, you might use 3 extra parity disks.

♦ **Features:**

- Provides **error detection and correction**
- **Very complex and rare** in practice

- Requires synchronized spindles (not practical for modern disks)

♦ **Used for:**

- Mostly **theoretical or experimental**; replaced by other RAID levels.
-

♦ **RAID Level 3 – *Byte-level Striping with Dedicated Parity Disk***

♦ **Concept:**

- Data is striped at the **byte level** across multiple disks.
- One **dedicated parity disk** stores parity information.
- If one disk fails, data can be **reconstructed** from parity.

♦ **Example (4 data + 1 parity):**

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4 (Parity)
A1	A2	A3	A4	P(A)
B1	B2	B3	B4	P(B)

♦ **Features:**

- High **transfer rate** (good for large sequential data)
- **Parity disk bottleneck** on writes
- Can **tolerate 1 disk failure**

♦ **Used for:**

- Applications requiring high throughput (video/audio editing)
-

♦ **RAID Level 4 – *Block-level Striping with Dedicated Parity Disk***

♦ **Concept:**

- Similar to RAID 3, but **striping at block level** instead of byte level.
- One disk stores parity for all data blocks.

♦ **Example (4 data + 1 parity):**

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4 (Parity)
A1	A2	A3	A4	P(A)
B1	B2	B3	B4	P(B)

♦ **Features:**

- Supports **independent reads** (multiple small requests in parallel)
- Write bottleneck on the parity disk
- **One disk failure tolerable**

♦ **Used for:**

- Rare today; replaced by RAID 5 due to parity bottleneck.

◆ RAID Level 5 – *Block-level Striping with Distributed Parity*

◆ Concept:

- Like RAID 4, but **parity is distributed** across all disks.
- Avoids the parity disk bottleneck by spreading parity blocks.

◆ Example (4 disks):

Disk 0	Disk 1	Disk 2	Disk 3
A1	A2	A3	P(A)
B1	B2	P(B)	B3
C1	P(C)	C2	C3
P(D)	D1	D2	D3

◆ Features:

- **Most popular** RAID level.
- Good balance of performance, capacity, and reliability.
- **Can survive one disk failure.**
- **Read and write operations are parallelized.**
- **Storage efficiency:** $(N-1)/N(N-1) / N(N-1)/N$

◆ Used for:

- File servers, web servers, large-scale data storage.

◆ RAID Level 6 – *Block-level Striping with Double Parity (P + Q)*

◆ Concept:

- Like RAID 5, but stores **two different parity blocks** (P and Q).
- Can tolerate **two disk failures** simultaneously.
- Uses **Reed–Solomon codes** for parity calculation.

◆ Example (5 disks):

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
A1	A2	A3	P(A)	Q(A)
B1	B2	P(B)	Q(B)	B3

◆ Features:

- Highest fault tolerance (2 disk failures)
- More CPU overhead for parity calculations
- **Slightly slower writes** than RAID 5

- **Storage efficiency:** $(N-2)/N(N-2) / N(N-2)/N$

♦ **Used for:**

- Mission-critical servers, large enterprise databases, backup systems.

♦ **Summary Table**

RAID	Technique	Striping Level	Redundancy	Fault Tolerance	Performance	Efficiency
0	Striping	Block	❌ None	0 disks	⚡ Fastest	100%
1	Mirroring	None	✅ Full copy	1 disk	⚡ Fast reads	50%
2	Hamming ECC	Bit	✅ ECC disks	1 disk	Slow	Low
3	Parity disk	Byte	✅ 1 parity disk	1 disk	Fast sequential	$(N-1)/N$
4	Parity disk	Block	✅ 1 parity disk	1 disk	Good reads	$(N-1)/N$
5	Distributed parity	Block	✅ Distributed	1 disk	⚡ Fast + Balanced	$(N-1)/N$
6	Dual parity	Block	✅ Double	2 disks	Good	$(N-2)/N$

♦ **Common Hybrid RAID Levels**

Level	Description	Example Use
RAID 0+1	Mirror of striped disks	Fast + redundant
RAID 1+0 (10)	Stripe of mirrored pairs	Even faster & more reliable
RAID 50	RAID 5 arrays striped together	Large data centers
RAID 60	RAID 6 arrays striped together	Very large and safe arrays

♦ **Summary in One Line per Level**

- **RAID 0** → Fastest, no protection.
- **RAID 1** → Mirrored for safety.
- **RAID 2** → Bit-level ECC (obsolete).
- **RAID 3** → Byte striping with parity (single parity disk).
- **RAID 4** → Block striping with parity (bottleneck on parity).
- **RAID 5** → Block striping + distributed parity (best general-purpose RAID).
- **RAID 6** → RAID 5 + double parity (survives 2 disk failures).

▼ Linear Hashing

<https://youtu.be/SiAHYSBkFTY?si=alkQzPoBrXFaNjpb>

▼ Questions

- ▼ How does double buffering reduce access time ?



Timing Analysis

Let:

- T_r = time to read a block from disk
- T_p = time to process a block

✗ Without double buffering:

Each block must wait for both read **and** process:

Total time per block = $T_r + T_p$

So for N blocks:

Total time = $N \times (T_r + T_p)$

✓ With double buffering:

After the first block is read, reading and processing **overlap**.

- The total time becomes roughly:

Total time $\approx T_r + (N - 1) \times \max(T_r, T_p)$

➡ Only the **slower of the two** (read or process) dominates after the pipeline is full.

Result: Reduced Effective Block Access Time

- The **CPU never waits** for the disk once the pipeline starts.
- The **disk never waits** for the CPU either.
- Effective time per block approaches **$\max(T_r, T_p)$** instead of $T_r + T_p$.

So, **block access time** — the average time between starting and completing processing of consecutive blocks — becomes much smaller

